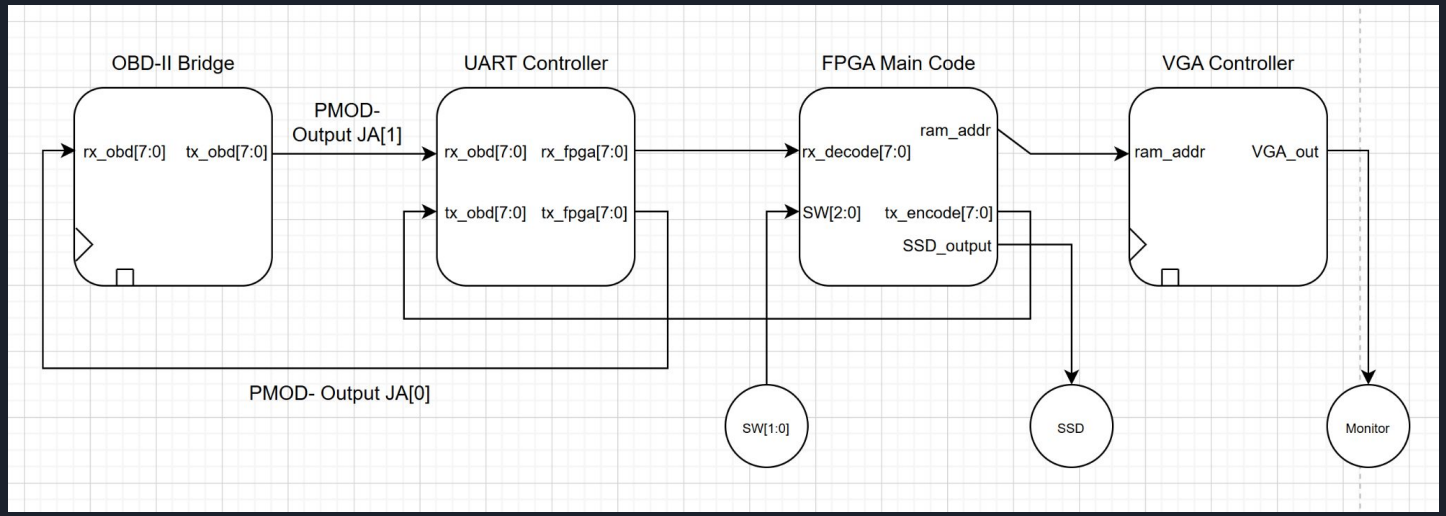
Final Project Design

By Abhi Boggavarapu, Trent Jones

Design Summary

Design Summary

Block Diagram: Overall Design



Design Summary

Key Decisions:

- Storing every Nth sample (reduce memory needed for storage)
- Reading-writing onboard DDR2 SDRAM (no external storage needed)
- Manually set baud rate through UART

Problem to be solved:

Many consumers and novice race teams do not have an affordable, quick way to gauge under-the-hood performance of their daily driver or modified hot rod. Our product aims to provide a quick and simple solution to display key statistics of engine performance from coolant temperature to RPM count.

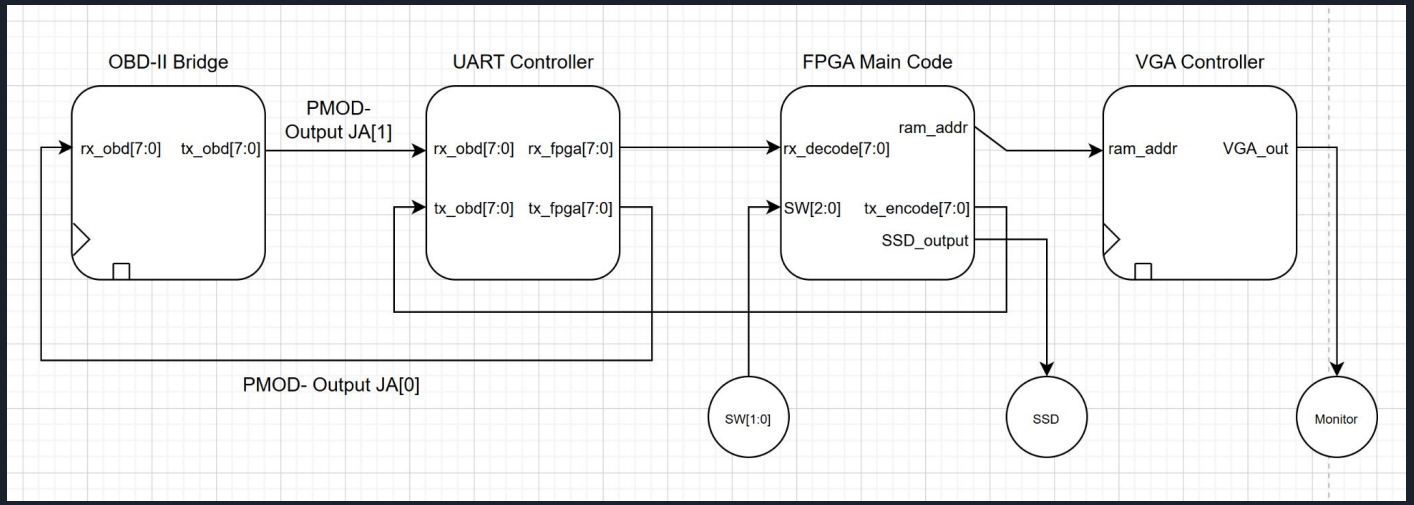
So… What’s Changed?

So... What's Changed?

- Taking every Nth sample can reduce the need for expanded storage and allow the use of on board RAM
- The design and protocol has been refined for reading and writing to DDR2 SDRAM, providing a clear basis for accessing and transmitting to VGA output
- The team has developed a timeline for work to be completed by both members in order to meet and exceed previously established deadlines

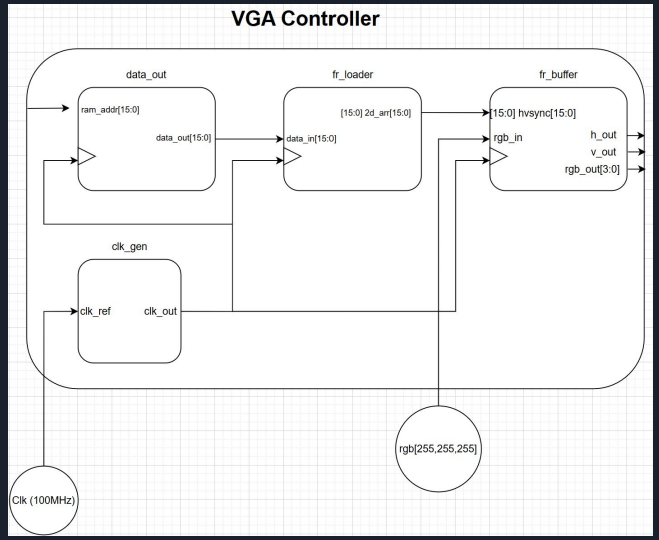
So… What's Changed?

- Updated Block diagram to show the PMOD uses
- Updated count on switches



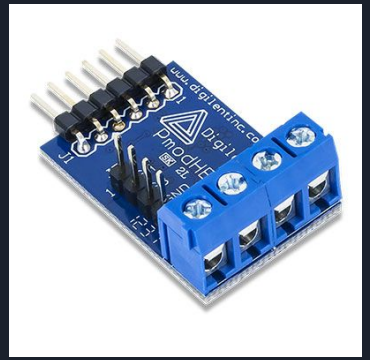
So… What's Changed?

The team has updated the VGA Controller Block Diagram to provide an in-depth view into the mechanics that will be used to read the address from the main FPGA code and output the data contained to the VGA monitor.



So… What's Changed?

- UART Bridge will be built using the PMOD header extension. This PMOD allows the FPGA to be able to have 4 working GPIO's. In the case of this project, we will use 2 of the header inputs as a RX and TX line.
- Although the FPGA already has a built in FPGA bridge, using the header extension allows the project to have a lot more control over the UART transmission.



So… What's Changed?

```
module memory(
    input clk,
    input [7:0] m_add, // Address for reading
    input [7:0] ead,   // Address for writing
    input [7:0] data_in, // Data to be written
    input ewr,         // Write enable
    input mrd,         // Read enable
    output reg [7:0] temp1 // Output data
);

    reg [7:0] memr [0:255]; // 256-byte memory array

    always @(posedge clk) begin
        if (ewr) begin
            memr[ead] <= data_in; // Write input data to the specified address
        end
        if (mrd) begin
            temp1 <= memr[m_add]; // Read data from the specified address
        end
    end

endmodule
```

The team has developed a basic memory read/write function once the data has been formatted from UART.

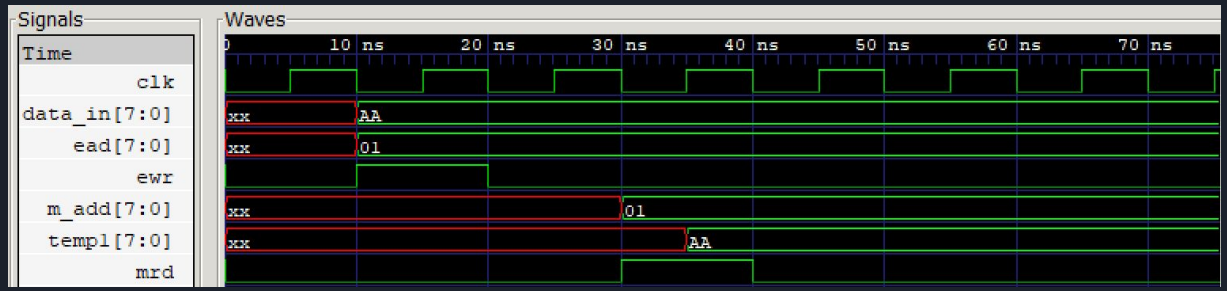
This will continue to be updated until it can store the quantity of data produced from the OBD-II module.

The data stored will continue to be format in a readable form for VGA transmission and output as the team works through the design process.

So… What's Changed?

Below is the waveform diagram of the aforementioned code through a self-designed testbench.

- Data_in: Data to be written when write is enabled (ewr)
- Ead: Address for writing
- Temp1: Output data when read function is enabled (mrd)
- M_add: Address for reading



Timeline

Timeline

	Trent (FPGA Main Code, VGA Controller)	Abhi (OBD-II Bridge, UART Controller)
Design Presentation	Research read/write in verilog, foundation for OBD to VGA com	Finish UART controller - Read Module + FIFO - Write Module + FIFO Buy Hardware
Preliminary Report	Finish storage component, shift to outputting to VGA	Be able to communicate with car via FPGA, Store ECU output in RAM FIFO
Final Report/Video	Complete OBD to VGA output and format UI and display, debug issues	Fully Integrate VGA and FPGA. Complete Project

Design Efficacy

Design Efficacy

- Proper efficient UART data transfers
- FPGA Resource Optimization
- Scalability of project
- Robust/Easy to understand Display onto Monitor

Debug Strategy

Debug Strategy

- Testbench Approach For General Modules/Communications
  - Take every module and test in blocks (UART controller would technically have about 7 or 8 testbenches)
  - Start small and build up
  - Test in iVerilog as much as possible for faster development
  - Only move to Vivado when moving to hardware synthesis and implementation
  - Testbenches are self-checking for quick analysis
- Testbench Approach for Memory
  - All of the above
  - Testing memory overflow/underflow
  - Using the ILA in Vivado to check correct placement of memory

Debug Strategy

We have come up with a relative process to debug/test our project throughout development:

- Divide and conquer approach to design
  - Allows for independent development and testing
- Each designated member will complete a method or program (depending on length and difficulty)
  - Once each defined section is completed, a testbench file with be constructed to determine any faults or missteps within said program.
- Allows for construction and debugging of code without a level of interdependence that would be detrimental to the completion of the project

Project Checkoff

Deliverables	Explanation	Commitment	Goal	Strech Goal
UART Read	Be able to properly read the incoming data from the car's ECU.	Properly read incoming data with 70% accuracy	Read data with 100% accuarcy and be able to diagnose errors if needed and alert system	Read data with 100% accuarcy and be able to diagnose errors if needed and alert system and user and be able to make an error detection system
UART Transmit	Be able to properly write the data from the FPGA to the Car.	ECU is able to properly read data string with 70% accuracy	Write data with 100% accuracy and have system alerts for if write goes wrong	Write data with 100% accuracy and have system alerts and error detection in place in case it happens
UART Read FIFO	This FIFO should be able to store enough of the read data until the FPGA side of the program handshakes data and properly requests it	FIFO is able to send data to the faster moving FPGA with proper handshake protocol	FIFO is able to send data to faster moving FPGA with proper handshake protocol with memory management in store	FIFO is able to send data to faster moving FPGA with proper handshake protocol and EFFCEIENT memory management is in place
UART TX FIFO	this FIFO should be able to take data from the fast moving FPGA side and properly provide the data to the slower UART controller	FIFO is able to take nessary data and provide it to the TX controller	FIFO is able to take nessary data and provide it to TX controller with memory management	FIFO is able to take nessary data and provide it to TX controller with memory management
FPGA MC Read Controller	This should be able to take data from the Read FIFO and convert that data into user readable information	Able to properly dissect information code read from ECU	Able to properly dissect information code read from ECU, utlizies lookup tabes and effeinct string parsing to maximize programming effeiency	Able to properly di ssect information code read from ECU, utlizies lookup tabes and effeinct string parsing to maximize programming effeiency as well as proper memory management
FGPA MC TX controller	This should be able to take user input on what code or information needs to be transmitted into the ECU	Able to parse user inputted information and create correct transmit code	Able to parse user inputted information and create correct transmit code as well as using efficent memory management te chniques and maximize LUT use	Able to parse user inputted information and create correct transmit code as well as using efficent memory management te chniques and maximize LUT use
FGPA RAM FIFO	This should be able to store and handshake data with the VGA controller when there is a data request.	Stores memory in DDRam and outputs correct address	Stores memory ini DDRam and outputs correct address with effeient storage and load protocols	Stores memory ini DDRam and outputs correct address with effeient storage and load protocols
VGA Data Handshake Modul	This module is meant for the handshake between the RAM FIFO and the processing modules of the VGA	Able to properly retrieve the data and process it whenever the frame rate is updated	Able to properly retrieve the data and process it whenever the frame rate is updated	Able to properly retrieve the data and process it whenever the frame rate is updated
VGA Frame Loader Module	This module is meant to make and process all of the VGA data and create the graph and other data that we want	Create the correct image vectors to display raw data	Create the correct image vectors to display graph of trip	Create the correct image vectors display graph of trip as well as average stats over trip
VGA Fram Buffer Module	This module loads the data from the FPGA to the monitor	Be able to display any sort of data	Be able to display graph data within specified frame rate	Be able to display graph data as well as average statistics data within frame rate and support capabilities for faster frame rate

Questions?